

KERLINK GATEWAY CONFIGURATION

- Power the Gateway and connect via Ethernet to network
- Open Advanced IP Scanner and scan the device

Advanced IP Scanner

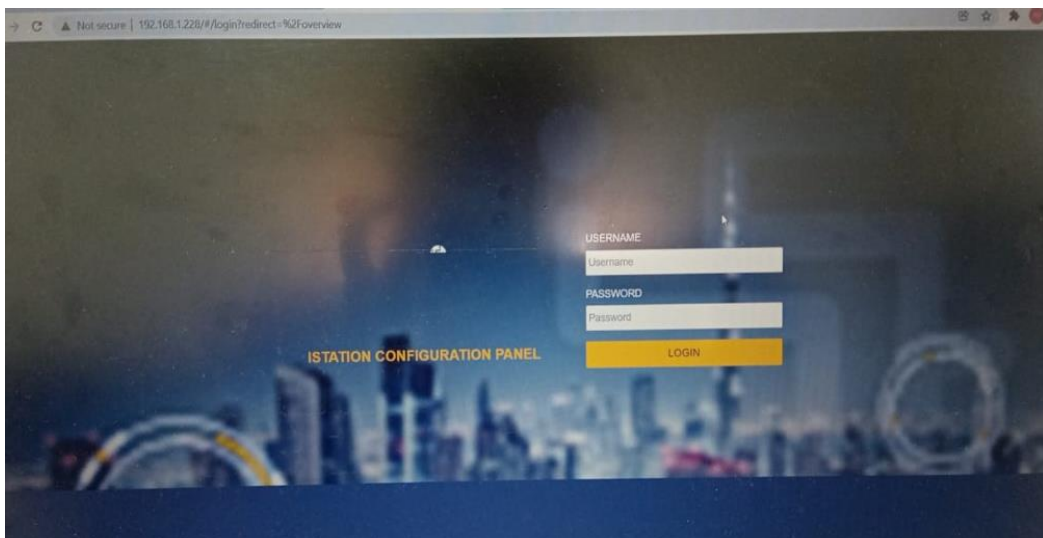
File View Settings Help

Scan

192.168.1.1-254

Status	Name	IP	Manufacturer	MAC address	Comments
	reliance.reliance	192.168.1.1		6C:DF:FB:2C:A8:39	
	LAPTOP-B8E19PRA	192.168.1.2	KERLINK	70:76:FF:05:02:03	
	Stores-PC	192.168.1.3	Dell Inc.	8C:EC:4B:9E:A6:EB	
	denvikpro-PC	192.168.1.4	ASUSTek COMPUTER I...	34:97:F6:9F:C8:8B	
	DenvikSPC	192.168.1.5	Dell Inc.	B8:AC:6F:97:35:91	
	Service	192.168.1.6	ASUSTek COMPUTER I...	38:D5:47:B2:BE:2B	
	192.168.1.8	192.168.1.8	Xiaomi Communicati...	20:A6:0C:36:40:9F	
	192.168.1.9	192.168.1.9		AC:F6:F2:35:43:AF	
	DenvikSPC	192.168.1.11	Dell Inc.	18:D8:F2:35:43:AF	
	DESKTOP-M9BT2TG	192.168.1.13		04:D4:C4:EF:29:92	
	production-PC	192.168.1.14	ASUSTek COMPUTER I...	60:A4:4C:CD:97:7B	
	192.168.1.16	192.168.1.16	KERLINK	70:76:FF:05:19:DF	
	192.168.1.17	192.168.1.17		6A:8D:4F:E4:C2:FC	
	192.168.1.19	192.168.1.19		EA:50:F4:05:CD:6D	
	192.168.1.21	192.168.1.21	vivo Mobile Communi...	34:E9:11:93:F8:2D	
	pc1-PC	192.168.1.22	ASUSTek COMPUTER I...	48:5B:39:E8:71:86	
	192.168.1.25	192.168.1.25		8A:F8:8B:18:69:91	
	DESKTOP-RF92AVG	192.168.1.29		E4:54:E8:BA:5D:07	
	192.168.1.30	192.168.1.30	Tenda Technology Co...	04:95:E6:B0:69:68	
	192.168.1.33	192.168.1.33		CE:4D:8F:B6:EA:4B	
	Mahadeer	192.168.1.34		04:D4:C4:B1:1C:1F	
	Denvik-Purchase	192.168.1.35	Dell Inc.	18:66:DA:3F:11:D8	

- Locate Kerlink and paste IP address in Chrome, Username: admin, Password: pwd4admin



- We have to configure Gateway bridge, so go to chirpstack.io/gateway-bridge/gateway/kerlink/

← → ↻ 🔒 chirpstack.io/gateway-bridge/gateway/kerlink/

ChirpStack Kerlink

service.downlink = 1700

After updating this configuration file, make sure to restart the `lorafwd` service:

```
monit restart lorafwd
```

Install ChirpStack Gateway Bridge

Find the latest package at <https://artifacts.chirpstack.io/vendor/kerlink/keros-gws/> and copy the URL to your clipboard. Then on the gateway use `wget` to download the package into a folder named `/user/.updates`. Example for `chirpstack-gateway-bridge_3.13.1-r1_klkgw.ipk`:

```
cd /user/.updates
wget https://artifacts.chirpstack.io/vendor/kerlink/keros-gws/chirpstack-gateway-bi
```

To trigger the gateway to install / update the package, run the following commands:

```
sync
kerosd -u
reboot
```

Please refer to the [Kerlink wiki](#) for more information about installing and updating packages.

Configure ChirpStack Gateway Bridge

Table of contents

- Kerlink KerOS based gateways
- Requirements
- SSH into the gateway
- Enable Kerlink CPF
- Configure Kerlink CPF
- Install ChirpStack Gateway Bridge
- Configure ChirpStack Gateway Bridge
- (Re)start and stop commands
- Kerlink IOT station

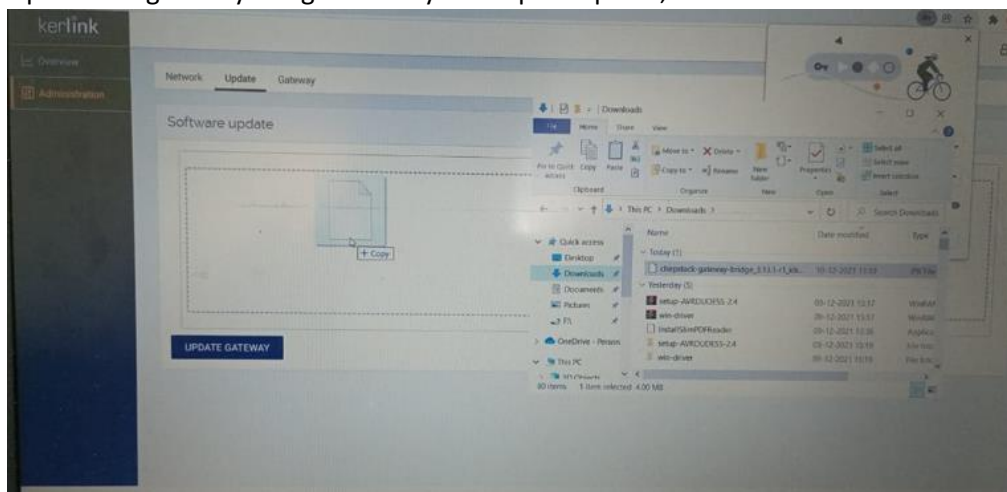
To connect the ChirpStack Gateway Bridge with your MQTT broker, you must update the

- Click on that link and download the latest release.

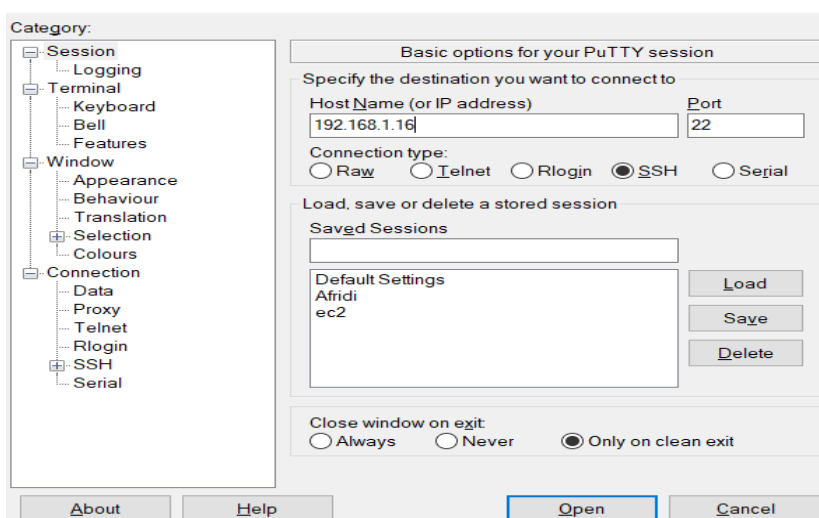
Index of /vendor/kerlink/keros-gws/

File Name ↓	File Size ↓	Date ↓
Parent directory/	-	-
chirostack-gateway-bridge_3.11.0-r1_klkow.ipk	4.0 MiB	2021-Jun-07 09:42
chirostack-gateway-bridge_3.12.0-r1_klkow.ipk	4.0 MiB	2021-Jul-21 13:56
chirostack-gateway-bridge_3.13.0-r1_klkow.ipk	4.0 MiB	2021-Aug-30 12:15
chirostack-gateway-bridge_3.13.1-r1_klkow.ipk	4.0 MiB	2021-Aug-31 13:34

- Upload the gateway bridge file to system update panel, then restart.



- Now paste Kerlink Gateway IP address in putty software for remote connection.



- Username: root, password: pdmk- "last four digit of gateway id" Eg: pdmk-0406F3
- Type this command - `klk_apps_config --activate-cpf --lms-server 3.21.48.255 --lms-dport 1700 --lms-uport 1700`

- Type the following command:
 - cd /etc/lorad/wifc/
 - cp EU868-IN.json /etc/lorad/lorad.json
 - reboot
- Login again to Kerlink Gateway, now we have to configure MQTT
- Type the following commands:
 - cd /etc/
 - vi lorafwd.toml
 - find topic **node**, it should contain "127.0.0.1"

```

192.168.1.228 - PuTTY
#
#delay.cooldown = 1000
[ gwmp ]
# The internet host where the gateway should connect. The node can be either a
# fully qualified domain name or an IP address (IPv4 or IPv6).
#
# Type: string
# Example: "myhost.example.com" or "123.45.67.89" or "2001:db8::1234"
# Default: "localhost"
#
node = "127.0.0.1"
# The GWMP services can be a service name (see services(5)) or an integer and,
# in this case, refers to a network port.
#
# The service where the gateway should push uplink messages.
#
# Type: string or integer
# Example: "https" or 1234
# Default: 0
#
service.uplink = 1700
# The service where the gateway should pull downlink messages.
#
# Type: string or integer
# Example: "https" or 1234
# Default: 0
#
service.downlink = 1700
# The heartbeat period. Used to keep the firewall open.
#
# Type: integer (in seconds)
# Example: 30
# Default: 10
#
period.heartbeat = 10
- lorafwd.toml 126/206 61%

```

- Now configure MQTT endpoint, do the following procedure.
 - cd chirpstack-gateway-bridge/
 - vi chirpstack-gateway-bridge.toml
 - Type your MQTT endpoint in **server** topic

```

192.168.1.228 - PuTTY
# Example: 0.0.0.0:1700 to listen on port 1700 for all network interfaces.
# This is the listener to which the packet-forwarder forwards its data
# so make sure the 'serv_port_up' and 'serv_port_down' from your
# packet-forwarder matches this port.
udp_bind = "0.0.0.0:1700"

# Integration configuration.
[integration]
# Payload marshaler.
#
# This defines how the MQTT payloads are encoded. Valid options are:
# * protobuf: Protobuf encoding
# * json: JSON encoding (easier for debugging, but less compact than 'protobuf')
marshaler="json"

# MQTT integration configuration.
[integration.mqtt]
# Event topic template.
event_topic_template="gateway/{{ .GatewayID }}/event/{{ .EventType }}"

# Command topic template.
command_topic_template="gateway/{{ .GatewayID }}/command/#"

# MQTT authentication.
[integration.mqtt.auth]
# Type defines the MQTT authentication type to use.
#
# Set this to the name of one of the sections below.
type="generic"

# Generic MQTT authentication.
[integration.mqtt.auth.generic]
# MQTT server (e.g. scheme://host:port where scheme is tcp, ssl or ws)
server="tcp://192.168.1.228:1883"

# Connect with the given username (optional)
username=""

# Connect with the given password (optional)
password=""
- chirpstack-gateway-bridge.toml 69/69 100%

```

- Now Gateway is configured.

- Download MQTT Explorer and Enter Host which you gave previous in Server Topic.

MQTT Connection

mqtt://ec2-3-21-48-255.us-east-2.compute.amazonaws.c...

Name

denvik lora

☒ Validate certificate

☐ Encryption (tls)

Protocol

mqtt://

Host

ec2-3-21-48-255.us-east-2.compute.amazonaws.com

Port

1883

Username

Password

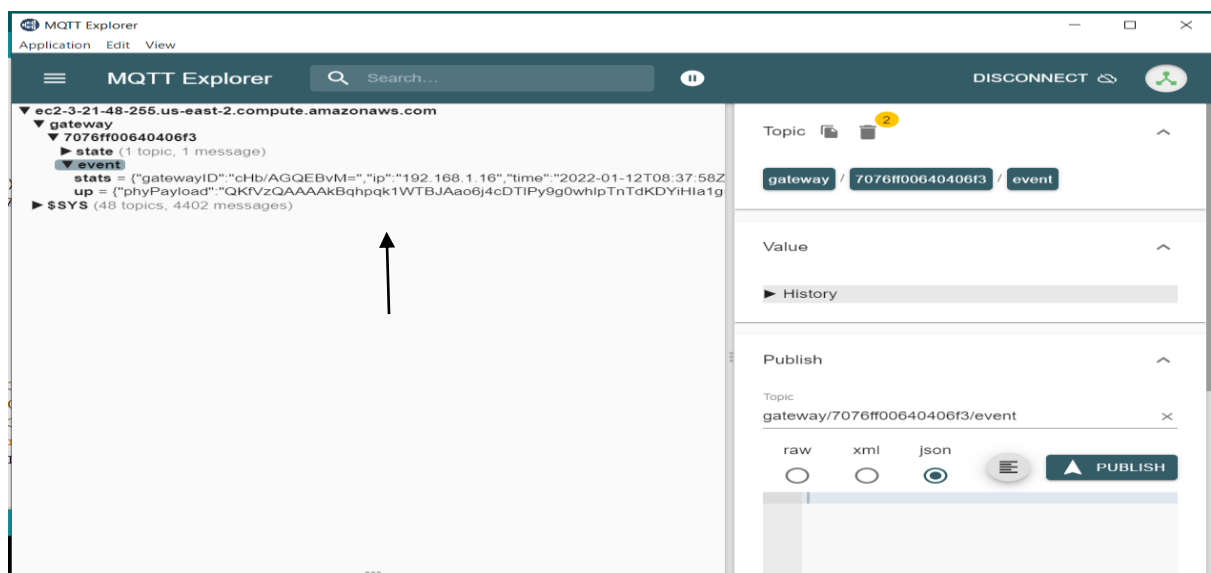
DELETE

ADVANCED

SAVE

CONNECT

- Now connect, then click **gateway** topic and in **Up** topic, you can able to view LoRa decrypted message.



LoRaWAN 1.0.x packet decoder

A frontend towards [lora-packet](#).

Base64 or hex-encoded packet (examples)

QKFVzQAAAAcBUx4T5iDnJZdxM47GLA1DaZoXNm15sTclxPngIAEQxg==

Decode

Secret NwkSKey (hex-encoded; optional)

fa3ecbf872a2310f8773fec8c872826

Secret AppSKey (hex-encoded; optional)

222d30e24f704abba68e73134caba3af

Specify the secrets if you want to validate the MIC and decrypt the payload. For an OTAA Join Request enter the AppKey in the AppSKey field; for an OTAA Join Accept see the explanation below. Secrets are sent to the server and might be stored in log files of RunKit.

```
Assuming base64-encoded packet
QKFVzQAAAAcBUx4T5iDnJZdxM47GLA1DaZoXNm15sTclxPngIAEQxg==

Message Type = Data
PHYPayload = 40A7D5CD000000701531E13E620E7259771338EC62C0D43699A17366D79B13725C4F9E0200110C6

( PHYPayload = MHDR[1] | MACPayload[...] | MIC[4] )
  MHDR = 40
  MACPayload = A7D5CD000000701531E13E620E7259771338EC62C0D43699A17366D79B13725C4F9E0
  MIC = 200110C6 (from packet)
      = 200110C6 (expected, assuming 32 bits frame counter with MSB 0000)

( MACPayload = FHDR | FPort | FRMPayload )
  FHDR = A7D5CD00000007
  FPort = 01
  FRMPayload = 531E13E620E7259771338EC62C0D43699A17366D79B13725C4F9E0 (from packet, encrypted)
              = 1F67F501E650159018807F0D8008EE10000C80A710C94107C1DE2 (decrypted)
```